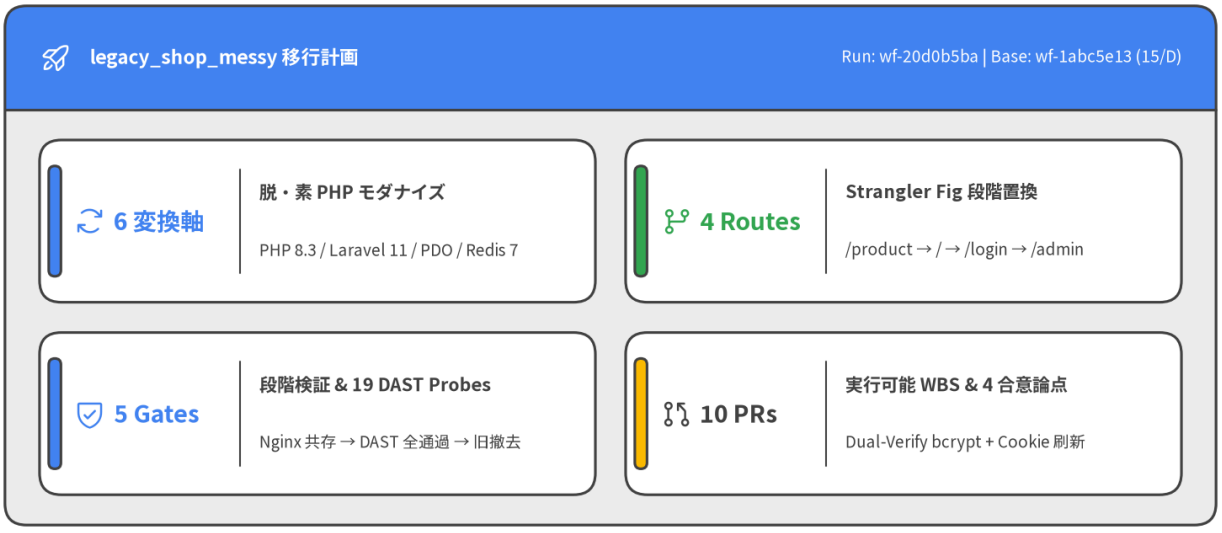
PNI マイグレーション実行計画レポート

本レポートは、ベースライン健全性監査（`wf-1abc5e13`: **15/100 Grade D**・ライブ Docker DAST `web_pentest` 19 プローブ中 10 件失敗）で未認証 RCE やマジックハッシュ認証バイパスが実証された手続き型「素 PHP」EC アプリケーション `legacy_shop_messy` に対する PNI 移行設計ワークフロー `php_legacy_shop_messy_migration` に基づき策定された、**PHP 8.3 + Laravel 11 + MySQL 8 PDO/Eloquent + Redis 7（Strangler Fig 段階カットオーバー）** の実行ブループリントである。



legacy_shop_messy Strangler Fig マイグレーション実行計画サマリカード

確定済み前提条件（Confirmed Assumptions）と非目標（Non-Goals）

- **移行モード / ロールアウト戦略**: `cross_stack_replacement` / `strangler_fig`（**Nginx リバースプロキシ配下のルート段階カットオーバー方式**）・互換ポリシー `strict`・曖昧性ポリシー `ask_human`
- **確定済み技術スタック**: **PHP 8.3**（`declare(strict_types=1)`）+ **Laravel 11**（Composer PSR-4 / PHPUnit 11 / PHPStan Level 6）+ **MySQL 8.0**（Eloquent / PDO プリペアドステートメント）+ **Redis 7**（暗号化セッション & サーバー側カート保存）

- **確定済みセキュリティ・DAST 回帰ガードレール**: 既存 URL と検索クエリ（`?q=`, `?cat=`）を維持しつつ、ライブ DAST で実証された OS コマンド実行（`PNI_RCE_CANARY_8842`）・マジックハッシュ認証突破（`'240610708'`）・LFI・Reflected XSS・SQLi・Cookie `unserialize` をルート置換と同時に根絶し、`.pni/pentests/examples/php/legacy_shop_messy/` の全 19 DAST プローブを CI ゲートとして再利用する。既存 MD5 会員は Dual-Verify 透過再ハッシュで無停止移行する。
- **非目標（Non-Goals）**: React / Vue 等の SPA フロントエンド分離および別言語リライトはスコープ外とし、自動エスケープを備えた Laravel Blade サーバーサイド描画へまず移行する。

Table of Contents

1. エグゼクティブサマリと Strangler Fig 移行戦略

2. As-Is 依存グラフ解析・移行 Wave・暗黙の外部契約

3. To-Be ターゲットアーキテクチャ設計（Strangler Fig & Modular Laravel 11）

4. 機械的変換（Rector Codemod）と手動ブロッカー分離計画

5. 「仕様かバグか」ステークホルダー意思決定ログ（Decision Log）

6. PR 単位の実行 WBS と検証・受け入れゲート

1. エグゼクティブサマリと Strangler Fig 移行戦略

`legacy_shop_messy` はベースライン監査（`wf-1abc5e13`: **15/100 Grade D**）およびライブ Docker DAST（`web_pentest`: **0/100 Grade D**、全 19 プローブ中 10 件失敗）において、`admin_backup.php` の未認証 OS コマンド実行（`PNI_RCE_CANARY_8842` 出力）・`login.php` の MD5 マジックハッシュ `'240610708'` による管理者認証バイパス・`?page=` LFI・Reflected XSS 2 箇所が実動実証された EOL `php:7.4-apache` スクリプト群である。インプレース修正や一括凍結リライトのリスクを回避するため、前段に Nginx ルーターを置き **1 エンドポイントずつ Laravel 11 へ無停止置換しつつ `.pni/pentests/` の 19 DAST プローブで段階検証する Strangler Fig（絞め殺しの木）戦略** を採る。

変換軸	As-Is (現状の課題)	To-Be (目標仕様)	移行アプローチ / リスク
言語・型安全	EOL PHP 7.4・strict_types ゼロ マジックハッシュ == 緩い比較	PHP 8.3 FPM (strict_types=1) Readonly Class / 厳密比較 ===	Rector + PHPStan Level 6 を 新規 Laravel 層に強制 [HIGH]
フレームワーク	composer.json 不在の素 PHP SQL と生 HTML echo が混在	Laravel 11 (FormRequest + Blade 自動エスケープ + Queue)	Nginx Strangler Fig ルート単位 段階カットオーバー [CRIT]
設計思想・構造	db.php <-> helpers.php 循環 $_GET/$_POST/$_SESSION 直参照	Modular Laravel 4層単方向構造 Http -> Action -> Port -> Infra	Phase 1 で共通インフラ層を構築 スーパーグローバル禁止 [HIGH]
DB・永続化	生 mysqli_query 文字列結合 DB_PASSWORD 定数直書き	MySQL 8 Eloquent / PDO Prepared .env 注入 + Migrations 管理	既存テーブル互換モデルで共存し SQLi を 100% 根絶 [CRIT]
キャッシュ・状態	unserialize($_COOKIE['shop_cart']) HttpOnly 欠落の生 $_SESSION 操作	Redis 7 暗号化セッション & RedisCartStore (TTL 7日 JSON)	Object Injection を即時遮断し サーバー側カートへ移行 [CRIT]
認証・保安統治	DAST 実証済み未認証 RCE・'240610708' 認証突破・LFI・XSS・TLS 無効 cURL	Dual-Verify bcrypt・Admin Gate Symfony Process 配列・TLS 強制	19 DAST プローブ回帰ゲートで 攻撃遮断を継続検証 [CRIT]

As-Is -> To-Be マルチ軸ギャップ分析マトリクス (全6変換軸)

5フェーズ段階的実行ゲート（Phase 0 → Phase 4）

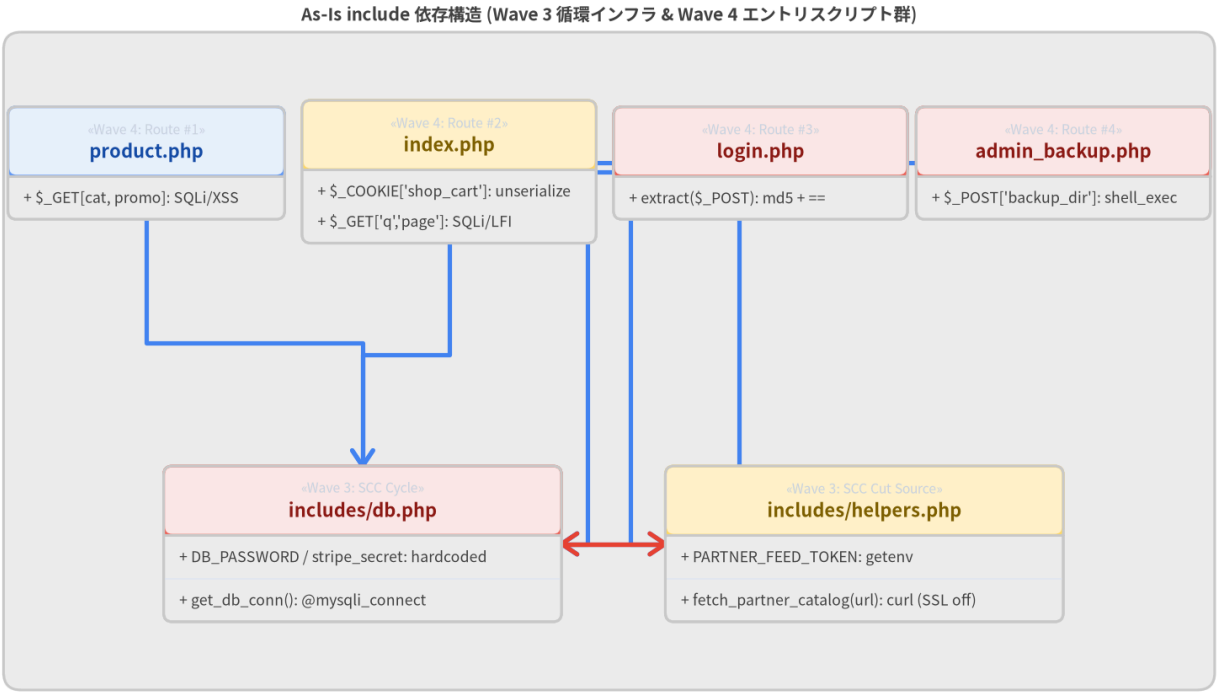


Strangler Fig 5フェーズ段階移行ロードマップ (Phase 0 -> Phase 4)

- **Phase 0（Nginx 共存基盤 & HTTP 特性テスト）**: 旧スクリプトを無変更のまま Nginx 背後へ配置し、Laravel 11（PHP 8.3）骨格と現行 URL の HTTP スナップショットテストを整備する。
- **Phase 1（循環解消 & Eloquent / Redis 7 基盤構築）**: `db.php <-> helpers.php` 循環依存を解消する形で PDO Repository・`RedisCartStore`・TLS 強制 HTTP クライアントを先行構築する。
- **Phase 2（閲覧・カート系カットオーバー）**: `/product.php` → `/index.php` の順に Nginx 振り分けを Laravel へ切り替え、Cookie `unserialize` と DAST 検出済みの Reflected XSS / `?page=` LFI / SQLi を排除する。
- **Phase 3（認証・管理ジョブ系カットオーバー）**: `login.php`（MD5 → bcrypt 透過再ハッシュ・マジックハッシュ `'240610708'` 排除）と `admin_backup.php`（未認証 `shell_exec` RCE 廃止・Symfony `Process` Job 化）を切り替える。
- **Phase 4（旧スクリプト完全撤去 & DAST 19 プローブ最終監査）**: 稼働安定確認後に旧 `.php` ファイル群を全削除し、`php:8.3-fpm-alpine` 非 root コンテナと `.pni/pentests/` 全 19 DAST プローブ PASS・Grade A を完遂する。

2. As-Is 依存グラフ解析・移行 Wave・暗黙の外部契約

`migration_analyze` による PHP `include_once` 依存グラフ解析の結果、共通インフラの `includes/db.php` と `includes/helpers.php` が相互 `include_once` する循環依存（Wave 3 SCC） を形成し、上位の4つのエントリスクリプト（`index.php`, `product.php`, `login.php`, `admin_backup.php`：Wave 4）がそれにぶら下がる構造が特定された。



As-Is 手続き型 include 依存グラフと db.php <-> helpers.php 循環切断ポイント

暗黙のスーパーグローバル入出力契約・秘密定数インベントリ（自動マスク済み）

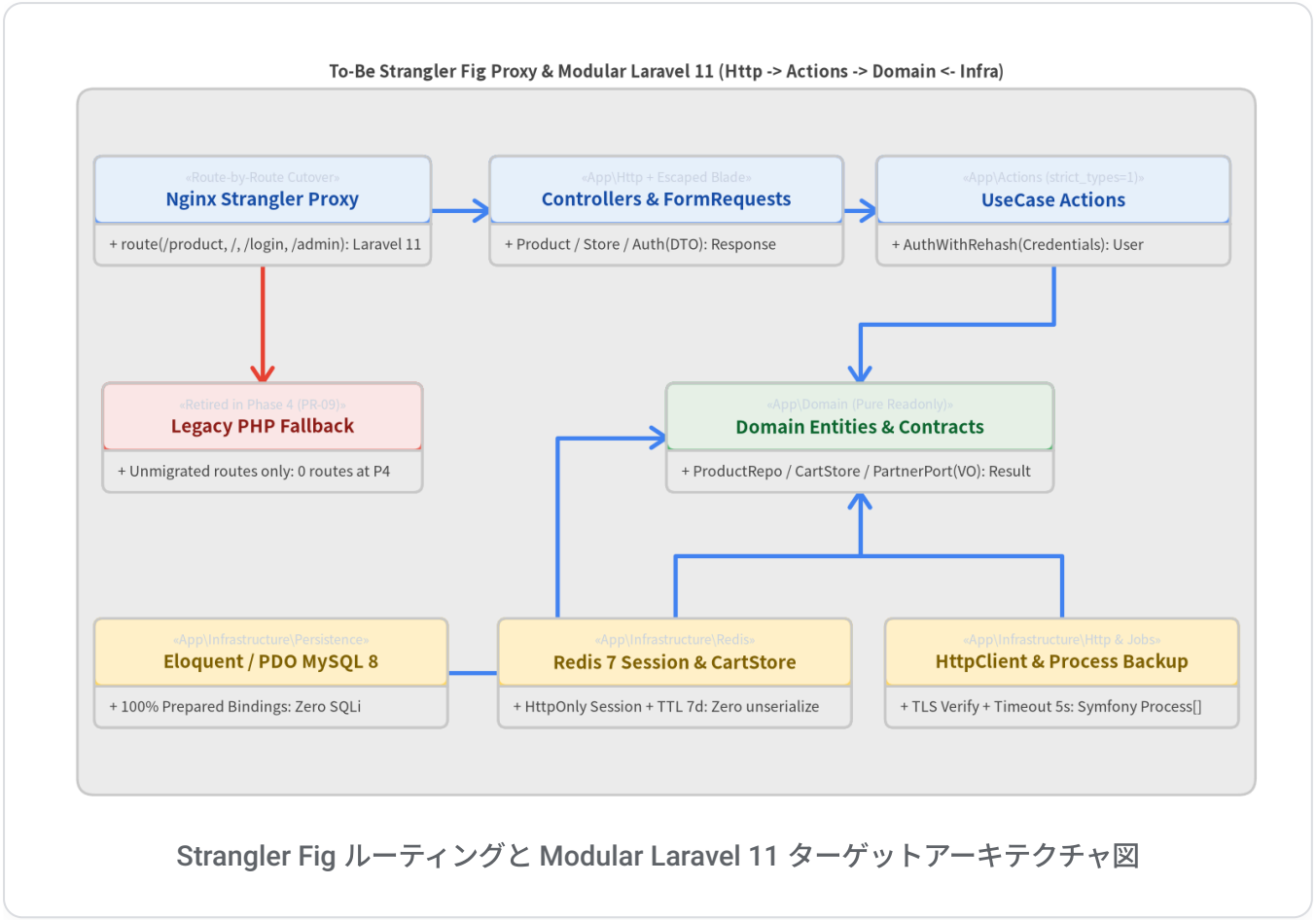
AST 解析により `$_GET` / `$_POST` / `$_COOKIE` / `define` / `getenv` から全スクリプトの暗黙パラメータ契約を抽出した（機密キー値は `<REDACTED>` マスク済み）。

エンドポイント / 定数	キー名 (方向)	既定値 / マスク	抽出箇所	Laravel 11 移行・無害化マッピング
GET /product.php	cat, promo_html (GET Input)	"featured" / "Special Sale"	product.php :L4, L14	ProductFilterRequest (?cat= 維持) + promo_code 許可リスト Blade 描画
GET /index.php	q, page (GET) shop_cart (Cookie)	null (未検証)	index.php :L4, L5, L24	?q= PDO 検索維持・?page= 固定 301・Cookie unserialize 廃止→RedisCartStore
POST /login.php	username, password (POST)	<REDACTED>	login.php :L7, L11	LoginRequest 厳密型検証 + extract 廃止 + Dual-Verify bcrypt 透過再ハッシュ
POST /admin_backup.php	backup_dir (POST Input)	null (CMDi 危険)	admin_backup :L4	ユーザーパス指定を廃止し固定 Storage ディスクへの非同期 BackupDatabaseJob 化
includes/db.php (define 定数)	DB_HOST, DB_USER, DB_PASSWORD	127.0.0.1 / <REDACTED>	includes/db.php :L4-L6	.env (DB_HOST / DB_USERNAME / DB_PASSWORD) + config/database.php へ外部化
includes/helpers.php (getenv / GET)	PARTNER_FEED_TOKEN / ?debug	null / ""	helpers.php :L6, L11	services.partner.token 設定注入 (Header 送信) + 本番 ?debug var_dump 完全削除

AST 抽出された暗黙のリクエスト/Cookie 契約と環境設定定数 (機密値マスク済み)

3. To-Be ターゲットアーキテクチャ設計（Strangler Fig & Modular Laravel 11）

前段の **Nginx Strangler Router** が移行済み URL パスを PHP 8.3-FPM（Laravel 11）へ、未移行パスのみをレガシーコンテナへ振り分けることで、ビッグバン凍結なしに段階移行する。Laravel 11 内部は `Http（Controller / FormRequest） -> Actions（UseCase） -> Domain Contracts（Port） <- Infrastructure（Eloquent / Redis / Http / Process）` の単方向アーキテクチャで構成する。



Strangler Fig ルーティングと Modular Laravel 11 ターゲットアーキテクチャ図

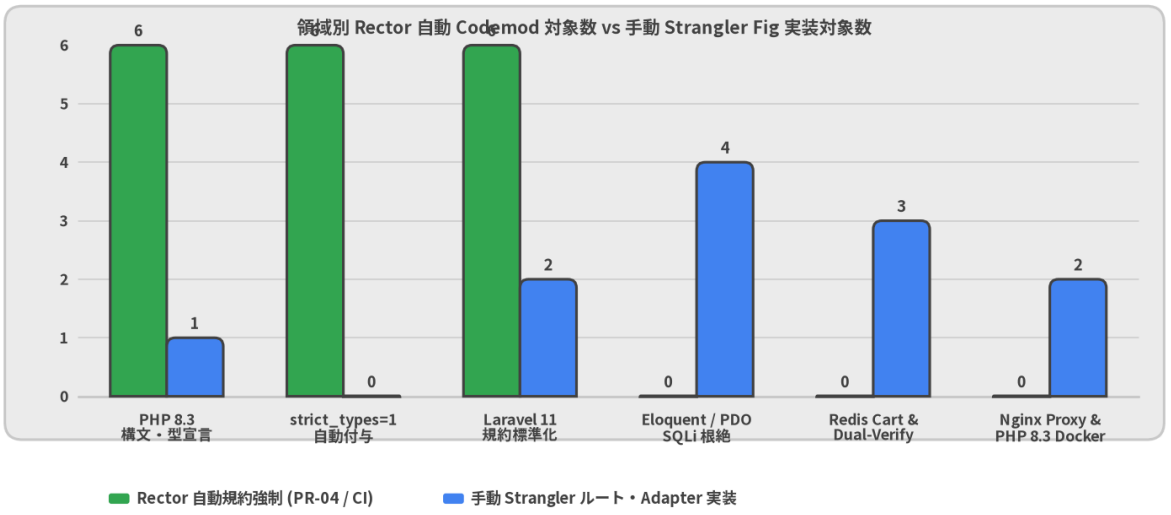
既存スクリプト・境界タッチポイントから Laravel 11 コンポーネントへのマッピング

As-Is ファイル	Strangler 順序	To-Be Laravel コンポーネント	根絶される CWE 脆弱性・状態分離
includes/db.php (@mysqli_connect)	Phase 1 基盤 (Infra)	config/database.php & Eloquent*Repository	DB_PASSWORD / Stripe 秘密鍵を .env 化し global $LAST_DB_ERR と @ 演算子を撤去。
includes/helpers.php (curl_init / debug)	Phase 1 基盤 (Infra)	HttpPartnerCatalogClient -> PartnerCatalogPort	SSL 検証無効化 (CWE-295) と URL トークン露出を廃止し TLS 強制 + 5秒 Timeout 化。
product.php (?cat= / ?promo_html=)	Phase 2 Route #1	ProductController + ProductFilterRequest	SQLi (CWE-89) を PDO バインドで解消し、XSS (CWE-79) を Blade 自動エスケープ化。
index.php (shop_cart / ?q= / ?page=)	Phase 2 Route #2	StorefrontController + RedisCartStore (TTL 7d)	Cookie unserialize (CWE-502) を Redis 化し ?page= 動的 include LFI (CWE-98) を封鎖。
login.php (extract / md5 / ==)	Phase 3 Route #3	AuthenticateUserWithRehash Action + Redis Session	extract (CWE-621) とマジックハッシュ == (CWE-697) を撤去し透過 bcrypt 再ハッシュ。
admin_backup.php (shell_exec)	Phase 3 Route #4	Admin\BackupController + BackupDatabaseJob	shell_exec (CWE-78) を廃止し Admin Gate + CSRF + Symfony Process 配列引数へ置換。

レガシー PHP 全6ファイルから Modular Laravel 11 への完全移行マッピング

4. 機械的変換（Rector Codemod）と手動ブロッカー分離計画

素 PHP から Laravel 11 へのクロススタック置換（`cross_stack_replacement`）では、Rector 単体でスパゲッティ HTML + 生 SQL スクリプトを自動的に MVC クラスへ分割することはできない。そのため、**ルート分割・Repository/Redis 設計は手動 PR（`PR-02` 〜 `PR-03`, `PR-05` 〜 `PR-08`）** で行い、切り出された PHP クラス群に対する `declare(strict_types=1)` **付与・PHP 8.3 型プロモーション・Laravel 11 規約標準化を Rector 自動パイプライン（`PR-04`）** に担わせる役割分担を徹底する。



変換領域別 Rector 自動規約適用数 vs 手動 Strangler 実装対象数

事前解消必須ブロッカー（BLK-01 〜 BLK-03）と Rector 適用レシピ

種別 / ID	レシピまたはブロッカー項目	対象範囲	実行タイミングと運用ルール
Blocker BLK-03 [HIGH]	composer.json / PSR-4 オートローダおよびテストハーネスの完全不在	プロジェクト全体	Phase 0 (PR-01): Composer・Laravel 11 骨格・PHPUnit 11・PHPStan Level 6 を初期化してから移行開始する。
Blocker BLK-01/02 [HIGH]	includes/db.php <-> helpers.php 双方向 include_once 循環依存	includes/ 配下 2ファイル	Phase 1 (PR-02, PR-03): Eloquent 設定と Partner HTTP クライアントを独立 Adapter として分離構築する。
Rector Set 1	Rector\Set\ValueObject\ LevelSetList::UP_TO_PHP_83	新規・抽出 PHP 全クラス	Phase 2 (PR-04) 以降 CI 常時強制: Readonly プロパティ、コンストラクタ昇格、PHP 8.3 標準構文へ統一。
Rector Rule 2	DeclareStrictTypesRector (declare(strict_types=1))	app/ config/ tests/ 全ファイル	Phase 2 (PR-04): 暗黙の型ジャグリング (0e... 緩い比較) が二度と混入しないよう全ファイル先頭へ機械挿入。
Rector Set 3	RectorLaravel\Set\ LaravelSetList::LARAVEL_110	Controller / Action / Model 全般	Phase 2 (PR-04): Laravel 11 のスリム Application 構成・型付き Casts メソッド規約を機械維持。

事前解消必須ブロッカー (BLK-01 - BLK-03) と Rector 自動変換レシピ一覧

5. 「仕様かバグか」ステークホルダー意思決定ログ（Decision Log）

レガシー PHP の脆弱性修正では、**セキュリティ無害化と既存ユーザー体験・ブックマーク互換のトレードオフ**が必ず発生する（例：既存データベース内の MD5 パスワードハッシュを単純に無効化すると全会員がログイン不能になり、移行瞬間の旧ブラウザ Cookie を `unserialize` せずにどう扱うかでカート挙動が変わる）。下表の4論点をステークホルダー承認ゲートとして管理する。



会員ロックアウトゼロを実現する MD5 -> bcrypt 透過再ハッシュ (DEC-01) フロー

ステークホルダー承認対象リスト（DEC-01 〜 DEC-04）

ID / 箇所	現行コードの観測挙動 (As-Is)	無断変更時の影響リスク	推奨方針 (Recommended) と選択肢
DEC-01 login.php :L7, L17	md5() とマジックハッシュ '0e4620...' を == 比較 (DAST で '240610708' による管理者突破を実証済み)。	DB カラムが 32文字 MD5 のまま bcrypt 専用照合へ切り替えると、全既存会員がログイン不能になる。	[推奨 Option A] Dual-Verify 透過移行: hash_equals で旧 MD5 照合成功時に bcrypt へ自動更新保存 [Option B] 全会員パスワード強制リセット
DEC-02 index.php :L4	$_COOKIE['shop_cart'] を直接 unserialize() してカート配列を復元している (CWE-502 RCE)。	Redis カートへ切り替えた瞬間、訪問中ブラウザに残る旧 serialize 形式 Cookie の未決済商品が消える。	[推奨 Option A] RCE 遮断を最優先し unserialize は完全禁止。旧 Cookie は失効・空カート初期化 [Option B] allowed_classes=false で数値 ID 移行
DEC-03 product.php :L14	GET パラメータ ?promo_html= をエスケープせずそのまま echo 出力 (DAST で Reflected XSS 実証済み)。	メルマガ等で HTML タグ付き URL を配布中の場合、Blade エスケープによりタグ文字列が露出する。	[推奨 Option A] 任意 HTML 入力を廃止し、許可済み ?promo_code= 識別子からサーバー側 Blade を選択 [Option B] クエリ廃止・DB 管理バナーへ統一
DEC-04 index.php:L24 helpers.php:L11	?page= に .php を連結して動的 include (DAST で LFI 露出実証)、?debug=1 で var_dump 出力。	?page=about 等の正規静的ページ URL をブックマークしている利用者が 404 エラーになる。	[推奨 Option A] ?debug は完全削除。?page= は固定 Enum (about/terms/contact) のみ 301 転送 [Option B] ?page= クエリを即時廃止

ルートカットオーバー前に合意必須のステークホルダー意思決定ログ (DEC-01 - DEC-04)

6. PR 単位の実行 WBS と検証・受け入れゲート

Strangler Fig の5フェーズ・4ルート置換を、1 PR ごとに独立テスト・即時切り戻し可能な **計 10 PR**（`PR-01` 〜 `PR-10`）に分割する。`PR-02`（MySQL PDO/Eloquent 基盤）と `PR-03`（Redis 7 セッション/カート & TLS HTTP Client）は `PR-01` 完了後に並列実装が可能である。

PR / Phase	規模 / 種別	PR タイトルと主要変更内容	依存 PR	必須検証コマンド (CI Gate)
PR-01 Phase 0	M / Manual	Composer / Laravel 11 + Nginx Strangler プロキシ基盤と HTTP 特性テスト構築	なし	composer test && phpstan analyse
PR-02 Phase 1 (W3)	M / Manual	db.php 秘密情報 .env 外部化と MySQL 8 Eloquent / PDO Repository Port 実装	PR-01	php artisan test --filter=EloquentRepo
PR-03 Phase 1 (W3)	S / Manual	Redis 7 セッション・RedisCartStore 実装と TLS 検証強制 Partner HTTP Client 構築	PR-01	php artisan test --filter=RedisCart
PR-04 Phase 2	XS / Codemod	Rector (PHP 8.3 / strict_types=1 / Laravel 11) 自動規約パイプライン導入	PR-02,03	rector --dry-run && phpstan analyse
PR-05 Phase 2 (R1)	S / Manual	Route #1: product.php の Laravel 移行 (?cat= PDO 化・promo Blade エスケープ)	PR-04	artisan test && pytest pentests -k product
PR-06 Phase 2 (R2)	M / Manual	Route #2: index.php の Storefront 移行 (Redis カート切替・?q= XSS/SQLi・LFI 根絶)	PR-05	artisan test && pytest pentests -k lfi
PR-07 Phase 3 (R3)	M / Manual	Route #3: login.php の Dual-Verify bcrypt 透過移行・'240610708' 突破根絶	PR-06	artisan test && pytest pentests -k auth
PR-08 Phase 3 (R4)	S / Manual	Route #4: admin_backup.php の Admin Gate + CSRF + Process Job 化 (RCE 根絶)	PR-07	artisan test && pytest pentests -k backup
PR-09 Phase 4	S / Manual	退役済み素 PHP スクリプト全6ファイルと Nginx 旧フォールバック設定の完全撤去	PR-08	composer test && phpstan --level=6
PR-10 Phase 4	S / Hybrid	php:8.3-fpm-alpine 非 root コンテナ・ CSP/HSTS ヘッダ・19 DAST CI 完成	PR-09	pni run-action web_pentest && trivy

Strangler Fig ルート順 PR 実行 WBS (PR-01 - PR-10) と自動検証コマンド

フェーズ完了受け入れ基準（Exit Criteria）と無停止ロールバック戦略

- **Phase 0 & Phase 1 完了条件**: Nginx リバースプロキシ経由で全既存 URL が透過動作し、`composer test` および `phpstan analyse --level=6` がエラーゼロで通過すること。
- **Phase 2 & Phase 3 動的 DAST 受け入れ条件**: `.pni/pentests/examples/php/legacy_shop_messy/` に収録された攻撃プローブ群（`backup_dir=; echo PNI_RCE_CANARY_8842` OS コマンド実行、`'240610708'` マジックハッシュ管理者突破、`?page=../../etc/passwd` LFI、`<script>` XSS 2 箇所、SQLi、未認証バックアップ実行）が **100% 防御成功（PASS）** すること。
- **Phase 4 最終受け入れゲート**: 旧素 PHP ファイル残存数 `0` 件、Trivy スキャンで `CRITICAL` / `HIGH` **脆弱性・秘密情報露出 0 件**、`web_pentest` 全 19 プローブ **19/19 PASS**、および PNI 健全性再診断（全 8 次元）で **Grade A（85点以上）** を達成すること。Strangler Fig 共存期間中（Phase 2〜3）に万一特定ルートで障害が発生した場合は、`PR-09`（旧ファイル撤去）マージ前であれば Nginx `location` 振り分け設定の即時リロードにより数秒で旧エンドポイントへ切り戻し可能である。